

Mocaverse Code Scanning — 回应报告

基于代码级确定性验证，对 Mocaverse code scanning 报告逐项核查的结果。

- 代码仓库: `mocachain/moca`
- 当前分支: `feat/e2e-validator-kind-migration`
- Module 路径: `github.com/evmos/evmos/v12`
- Go 版本: `go 1.23.6`
- 核查日期: 2026-04-04

一、Critical CVE 核查

1.1 CVE-2024-32644 (CRITICAL) — 预编译中 stateDB.Commit()

项目	内容
原报告描述	预编译在执行期间调用 <code>stateDB.Commit()</code> ，导致部分状态持久化，可无限铸币
结论	不存在该漏洞 — 攻击路径不成立

确定性证据（三层证明）：

1. 接口隔离 — `go-ethereum/core/vm/interface.go:27-80` 定义了 `vm.StateDB` 接口，该接口不包含 `Commit()` 方法。所有预编译通过 `vm.StateDB`（类型为 `vm.StateDB`）访问状态数据库。Go 编译器在编译期即拒绝通过该接口调用 `Commit()`。
2. 无类型断言绕过 — 搜索全部 12 个预编译包（`x/evm/precompiles/*`），查找 `.(*StateDB)` 或 `statedb.StateDB` 等类型断言 — 零匹配。没有任何预编译代码将 `vm.StateDB` 转型为具体的 `*statedb.StateDB` 类型。
3. 预编译中无 `Commit()` 调用 — 搜索整个 `x/evm/precompiles/` 目录中的 `.Commit()` 调用 — 零匹配。唯一的生代码中的 `stateDB.Commit()` 位于 `x/evm/keeper/state_transition.go:416-421`，在 `evm.Call()/evm.Create()` 完全返回之后才被调用：

```
if commit {
    if err := stateDB.Commit(); err != nil {
        return nil, errorsmod.Wrap(err, "failed to commit stateDB")
    }
}
```

1.2 CVE-2024-37153 (CRITICAL) — ICS-20 转账不扣除合约余额

项目	内容
原报告描述	智能合约发起 ICS-20 转账时不扣除合约余额，可通过 IBC 无限转账

项目	内容
结论	不存在该漏洞 — 攻击面不成立

确定性证据（两层证明）：

1. **不存在 ICS-20 Transfer 预编译** — `app/app.go:1408-1474` 中注册的 12 个预编译为：`bank` `authz` `gov` `payment` `permission` `staking` `distribution` `storage` `virtualgroup` `storageprovider` `slashing` `erc20`。没有注册 `transfer/ibc/ics20` 预编译，`x/evm/precompiles/` 下也不存在 IBC 转账相关的预编译目录。
2. **EOA 限制阻断间接路径** — `authz` 预编译理论上可通过 `Exec()` 方法执行 `MsgTransfer`。但全部 11 个具有写操作的预编译的所有写方法都强制要求 `evm.Origin == contract.Caller()`：

```
if evm.Origin != contract.Caller() {
    return nil, types.ErrInvalidCaller
}
```

当智能合约调用预编译时，`contract.Caller()` 是合约地址，`evm.Origin` 是 EOA 地址——两者不同，调用被拒绝。只有 EOA 直接调用才能通过此检查，而 EOA 调用会正确从 EOA 的 Cosmos bank 余额中扣除。

已逐一验证全部 11 个预编译的 `tx.go` 文件：

预编译	EOA 检查次数	文件
bank	2	<code>x/evm/precompiles/bank/tx.go</code>
authz	3	<code>x/evm/precompiles/authz/tx.go</code>
gov	5	<code>x/evm/precompiles/gov/tx.go</code>
staking	5	<code>x/evm/precompiles/staking/tx.go</code>
distribution	7	<code>x/evm/precompiles/distribution/tx.go</code>
payment	4	<code>x/evm/precompiles/payment/tx.go</code>
storage	35	<code>x/evm/precompiles/storage/tx.go</code>
virtualgroup	10	<code>x/evm/precompiles/virtualgroup/tx.go</code>
storageprovider	1	<code>x/evm/precompiles/storageprovider/tx.go</code>
slashing	1	<code>x/evm/precompiles/slashing/tx.go</code>
erc20	2	<code>x/evm/precompiles/erc20/tx.go</code>
permission	0（无写方法）	<code>x/evm/precompiles/permission/tx.go</code>

1.3 GHSA-3fp5-2xwh-fxm6（CRITICAL） — 非原子 $A \rightarrow B \rightarrow A$ 状态转换

项目	内容
----	----

项目	内容
原报告描述	非原子 $A \rightarrow B \rightarrow A$ 状态转换绕过持久化，同属无限铸币类漏洞
结论	架构缺陷存在，但当前不可被利用（被 EOA 限制阻断）

漏洞模式在架构层面确认存在（4 项条件全部成立）：

1. $A \rightarrow B \rightarrow A$ 跳过逻辑已确认 — `x/evm/statedb/statedb.go:507-509` 中，`Commit()` 在 `dirtyStorage[key] == originStorage[key]` 时跳过写入：

```
for _, key := range obj.dirtyStorage.SortedKeys() {
    value := obj.dirtyStorage[key]
    // Skip noop changes, persist actual changes
    if value == obj.originStorage[key] {
        continue
    }
    s.keeper.SetState(s.ctx, obj.Address(), key, value.Bytes())
}
```

2. 预编译 `commit()` 立即执行且不可逆 — 全部 12 个预编译遵循相同模式：`CacheContext()` → 执行 → `commit()`。一旦 `commit()` 执行，Cosmos 状态变更（如 bank 余额转账）即合并到父级上下文，无法撤销。

```
ctx, commit := c.ctx.CacheContext()
snapshot := evm.StateDB.Snapshot()
```

```
commit()
return ret, nil
```

3. EVM journal 不追踪 Cosmos 状态 — `x/evm/statedb/journal.go` 仅追踪 EVM 层面的变更（余额、nonce、代码、存储）。`RevertToSnapshot()` 无法回滚预编译 `commit()` 已写入的 Cosmos KV store 变更。
4. 无原子性包装机制 — 在整个代码库中搜索 `RevertMultiStore.RunAtomicWriteAccessList` — 零匹配。

为何当前不可被利用：

该漏洞的利用需要智能合约调用预编译后在同一调用栈内触发 revert。但所有预编译写方法都强制 `evm.Origin == contract.Caller()`，阻断了智能合约调用者。当 EOA 直接调用预编译时，没有上层合约能在预编译成功后触发 revert。

风险评估：安全性完全依赖 `evm.Origin != contract.Caller()` 这一单层检查。任何未来新增预编译若遗漏此检查，漏洞将立即可被利用。根因（Cosmos/EVM 状态非原子性）未修复。

1.4 GHSA-mjfq-3qr2-6g84 (HIGH) — 低 gas 导致预编译部分执行

项目	内容
原报告描述	低 gas limit 导致预编译部分执行不回滚，可导致资金盗取或验证器宕机
结论	架构缺陷存在，但当前不可被利用（被 EOA 限制阻断）

漏洞模式确认存在（3 项条件）：

1. **缺少修复符号** — 在整个代码库中搜索 `RevertMultiStore.HandleGasError.RunAtomic` — 零匹配 `cosmos/evm` 标准修复未被应用。
2. **`RequiredGas()` 可返回 0** — 全部 12 个预编译在方法解析失败或方法未注册时返回 0。例如 `x/evm/precompiles/bank/contract.go:70-105`：

```
func (c *Contract) RequiredGas(input []byte) uint64 {
    method, err := GetMethodByID(input)
    if err != nil {
        return 0
    }
}
```

```
default:
    return 0
}
```

3. **EVM `RevertToSnapshot` 无法回滚 Cosmos 状态** — 与 GHSA-3fp5 根因相同。若外层 EVM 帧在预编译 `commit()` 执行后 revert，Cosmos 状态变更将持久保留而 EVM 状态被回滚。

理论攻击场景：

```
Contract B 调用 Contract A
├─ Contract A 调用 precompile.Delegate(gas=600000)
│   ├─ RequiredGas() = 600000, UseGas(600000) ✓
│   ├─ Run() 执行, commit() 写入 Cosmos 质押状态
│   └─ 返回时 gas=0
├─ Contract A 下一步操作 OOG
├─ Contract B 捕获 revert: evm.StateDB.RevertToSnapshot()
└─ EVM 状态回滚 ✓, 但 Cosmos 质押状态仍然保留 ✗
```

为何当前不可被利用：与 GHSA-3fp5 相同 — EOA 限制 (`evm.Origin != contract.Caller()`) 阻止智能合约调用预编译写方法。Contract A 无法调用 `precompile.Delegate()`，因为 `evm.Origin` (EOA) $\neq$ `contract.Caller()` (Contract A)。

二、缺失安全特性核查

2.1 单笔 Cosmos 交易仅允许一笔 EVM 交易

项目	内容
原报告描述	无 EVM 交易批量限制，允许批量攻击
结论	确认缺失

`app/ante/evm/setup_ctx.go:56-58` 注释明确处理"多条 eth msg"：

```
// Reset transient gas used to prepare the execution of current cosmos tx.
// Transient gas-used is necessary to sum the gas-used of cosmos tx, when it contains multiple eth msgs.
esc.evmKeeper.ResetTransientGasUsed(ctx)
```

`EthEmitEventDecorator` (`setup_ctx.go:78-91`) 对 `tx.GetMsgs()` 做 for 循环处理，无 `len(msgs) == 1` 校验。

2.2 原子预编译执行 (RevertMultiStore)

项目	内容
原报告描述	缺少 <code>RevertMultiStore()</code> ，存在根本性安全缺口
结论	确认缺失

在整个代码库中搜索 `RevertMultiStore` — 零匹配。预编译使用 `CacheContext` + EVM `Snapshot/RevertToSnapshot`，但这两套系统相互独立。Cosmos `commit()` 一旦执行即不可逆，而 EVM `RevertToSnapshot` 仅回滚 journal 条目。

2.3 IBC 速率限制中间件

项目	内容
原报告描述	无 IBC 速率限制中间件，无法防止快速资金流失
结论	确认缺失

`app/app.go:604-607` 的 IBC transfer 栈：

```
var transferStack porttypes.IBCModule
transferStack = transfer.NewIBCModule(app.TransferKeeper)
transferStack = erc20.NewIBCMiddleware(app.Erc20Keeper, transferStack)
```

仅包含 `erc20` 中间件，无 `rateLimit` 中间件。代码库中的 `rate_limit` 引用均属于 `storage` 模块的 bucket 流控 (`x/storage/`)，与 IBC 无关。

2.4 EVM Pre-blocker 非确定性状态变更

项目	内容
原报告描述	EVM pre-blocker 中存在非确定性状态变更，有共识分叉风险
结论	不存在该风险 — EVM 未注册 PreBlocker

app/app.go:778-780 — 仅 upgrade 模块设置了 PreBlocker：

```
app.mm.SetOrderPreBlockers(
    upgradetypes.ModuleName,
)
```

EVM BeginBlock 仅调用 WithChainID(ctx)，EndBlock 仅从 transient store 读取 bloom 并发出事件（x/evm/keeper/abci.go:25-42），无非确定性来源。

三、缺失 EVM 特性核查

3.1 EIP-7702

项目	内容
原报告描述	缺少 EIP-7702（EOA 账户代码设置/委托）
结论	确认缺失

在整个代码库中搜索 7702 和 eip7702 — 零匹配。SetCode 仅存在于 StateDB/keeper 的常规合约代码存储中，非 EIP-7702 的 delegation 机制。

3.2 自定义 EVM 应用侧内存池

项目	内容
原报告描述	缺少自定义 EVM 应用侧内存池
结论	确认缺失

app/app.go:358-364 使用 NoOpMempool + 默认 proposal 处理器：

```
baseAppOptions = append(baseAppOptions, func(app *baseapp.BaseApp) {
    mempool := mempool.NoOpMempool{}
    app.SetMempool(mempool)
    handler := baseapp.NewDefaultProposalHandler(mempool, app)
    app.SetPrepareProposal(handler.PrepareProposalHandler())
    app.SetProcessProposal(handler.ProcessProposalHandler())
})
```

四、总结

#	问题	严重性	结论	说明
1	CVE-2024-32644 (Precompile Commit)	CRITICAL	不存在	<code>vm.StateDB</code> 接口不含 <code>Commit()</code> 方法，预编译无法调用。编译期类型系统保证。
2	CVE-2024-37153 (ICS-20 余额)	CRITICAL	不存在	不存在 ICS-20 预编译；全部写操作预编译均强制 EOA 限制。
3	GHSA-3fp5-2xwh-fxm6 (A → B → A 非原子)	CRITICAL	架构缺陷存在，当前不可利用	Cosmos/EVM 状态非原子性已确认。当前被全部预编译写方法的 EOA 限制阻断。单层防御。
4	GHSA-mjfq-3qr2-6g84 (低 gas 部分执行)	HIGH	架构缺陷存在，当前不可利用	与 #3 根因相同。 <code>RevertMultiStore/RunAtomic</code> 未应用。当前被 EOA 限制阻断。
5	单笔 EVM 交易限制	安全特性	确认缺失	AnteHandler 处理多条 <code>MsgEthereumTx</code> 无数量限制。
6	原子预编译执行 (RevertMultiStore)	安全特性	确认缺失	代码库中 <code>RevertMultiStore</code> 零匹配。
7	IBC 速率限制中间件	安全特性	确认缺失	Transfer 栈仅有 <code>erc20</code> 中间件，无速率限制。
8	EVM PreBlocker 非确定性	安全特性	不存在该风险	EVM 无 PreBlocker。BeginBlock/EndBlock 为确定性操作。
9	EIP-7702	EVM 特性	确认缺失	<code>7702/eip7702</code> 零匹配。
10	自定义 EVM 内存池	EVM 特性	确认缺失	使用 <code>NoOpMempool</code> + 默认处理器。

风险评估

当前风险：低 — 两个架构缺陷 (GHSA-3fp5、GHSA-mjfq) 确实存在，但当前被 EOA 限制 (`evm.Origin != contract.Caller()`) 完全阻断。该检查覆盖全部 11 个有写操作的预编译合约的全部 75 个写方法入口。

潜在风险：高 — EOA 检查是抵御关键状态不一致攻击的**唯一**防线。这是脆弱的单层防御：

- 任何新增预编译遗漏 EOA 检查 → 漏洞立即可利用
- 任何重构修改该检查 → 漏洞立即可利用
- 根因 (Cosmos `commit()` 不被 EVM journal 追踪) 未修复

优先级建议

P0 (尽快处理)：

- 从 Evmos v17+ 或 cosmos/evm 移植 `RevertMultiStore/RunAtomic`，从架构层面修复 Cosmos/EVM 原子性缺口，消除对 EOA 限制作为唯一防线的依赖

P1（短期内处理）：

- 在 AnteHandler 中添加单笔 EVM 交易限制
- 在 IBC transfer 栈中添加速率限制中间件

P2（按路线图推进）：

- EIP-7702 支持
- 自定义 EVM 应用侧内存池